

RAG — a brain of your *own* data.

A field workbook for the Lumière build. Four personas. Five build steps. Three reference sheets. Two prompts to take home and rebuild it from scratch.

Table of *Contents.*

Orientation

The Lumière Knowledge Base — Key Facts at a Glance

The Four Personas — Pick One

Persona 1 — Last-minute Birthday

- Build A — Feed the Brain
 - Build B — Make It Cite
 - Build C — Make It Refuse
 - Build D — Observe and Design
 - Build E — Stress and Ship
-

Persona 2 — Allergy Parent

- Build A — Feed the Brain
 - Build B — Make It Cite
 - Build C — Make It Refuse
 - Build D — Observe and Design
 - Build E — Stress and Ship
-

Persona 3 — Corporate Bulk

- Build A — Feed the Brain
 - Build B — Make It Cite
 - Build C — Make It Refuse
 - Build D — Observe and Design
 - Build E — Stress and Ship
-

Persona 4 — Delivery Edge-case

- Build A — Feed the Brain
- Build B — Make It Cite

Build C — Make It Refuse
Build D — Observe and Design
Build E — Stress and Ship

Reference Sheet 1 — Grounding Prompt Library

The grounding prompt (Build B)
The refusal line (Build C)
Combined prompt (copy-paste ready for Steps B and C together)
Sharpening tips

Reference Sheet 2 — The RAG Dial

Reference Sheet 3 — Knowledge Base Design Sheet

Fillable sheet
Worked example — Last-minute Birthday persona

Troubleshooting

Build Lumière yourself — two prompts, two stages

Before you start
Stage 1 — A grounded bakery bot
Stage 2 — Make it agentic
What the tools actually do — read this before you wonder
Architecture — both stages, side by side
What you have, end to end

What's Next — Session 5

RAG: Giving Agents a Brain of Your Own Data

What you build today: A knowledge agent grounded on the Lumière Bakery document — one that answers from the file, names its source, and says "I don't have that in my knowledge base" when the answer isn't there.

What you take home: A completed Knowledge Base Design Sheet — a reusable recipe for grounding any document at work.

Orientation

All learners work from the same document: [Lumiere_KB.md](#). You'll find the download link in the calendar invite.

Pick a persona below that fits your mindset today. Your persona shapes the questions you ask — the document is the same for everyone.

No Project? Attach [Lumiere_KB.md](#) to a fresh chat — same result.

Reference sheets at the end of this workbook:

- Grounding Prompt Library
 - The RAG Dial
 - Knowledge Base Design Sheet
-

The Lumière Knowledge Base — Key Facts at a Glance

Use this as a quick orientation before you start. Everything here is in `Lumiere_KB.md` ; the agent you build today will answer questions about all of it.

- **Branches:** Bandra West (Mon–Fri 7:30am–8:30pm), Andheri West (Mon–Fri 8am–8pm), Powai (daily 8am–9pm)
 - **Custom cakes:** Minimum 48h lead time; 72h for cakes over 3kg or with fondant; same-day orders are never accepted; 50% advance required
 - **Eggless option:** Available for all standard cake flavours at +₹80 per cake
 - **Eggless lead time:** Same as standard — 48h minimum, 72h for fondant or over 3kg
 - **Allergens:** The kitchen is NOT nut-free; Lumière cannot guarantee any item is allergen-free
 - **Delivery:** 8km radius from each branch; Mumbai only (no Gurgaon in Version 1 of the document)
 - **Current offer:** 15% off celebration cakes ordered 3 or more days ahead — code LUMIERE15
 - **Eggless cakes and the LUMIERE15 offer:** The offer applies to eggless celebration cakes too, as long as the 3-day minimum is met
-

The Four Personas — Pick One

#	PERSONA	IN BRIEF
1	Last-minute Birthday	Needs a 2kg eggless cake by Saturday — never ordered from Lumière before
2	Allergy Parent	Child has a severe nut allergy; needs to know what's safe before ordering
3	Corporate Bulk	Organising an 80-person office event in Andheri, Friday delivery
4	Delivery Edge-case	Lives in Sector 90, Gurgaon; heard Lumière is expanding

Work through **Builds A–E** on your chosen persona page. The build steps carry the same heading on every persona page — when the facilitator calls "Build C," find it in the same place on your page.

Persona 1 — Last-minute Birthday

You need a 2kg eggless chocolate cake for this Saturday. You've heard Lumière is good but you've never ordered from them. You also want to know whether the eggless option adds to the price and whether you can still get the 15% offer code for the eggless version.

Your starter document: `Lumiere_KB.md`

Build A — Feed the Brain

Upload `Lumiere_KB.md` to your Claude Project (or attach to a fresh chat). Then ask:

"Can I get a 2kg eggless chocolate cake for this Saturday?"

Log the answer here:

Build B — Make It Cite

Open Reference Sheet 1 at the end of this workbook. Copy the **Grounding Prompt** and paste it into your Project's custom instructions (or as the first message of your chat). Then re-ask your Build A question.

The answer should now end with a real **(Source: ...)** line naming a specific heading — not just "the document."

Paste the source line the agent returned:

(Source: _____)

Build C — Make It Refuse

Ask your agent a question that is outside the document:

"Can you match a competitor's price on a custom eggless cake?"

Before adding the refusal line — what did the agent say?

Open Reference Sheet 1. Copy the **Refusal Line** and add it to your grounding prompt. Re-ask the same question.

After — what did the agent say?

Build D — Observe and Design

Facilitator demo. Watch the live screen and fill in the KB Design Sheet chunk/tag fields at the end of this workbook based on what you observe.

Key thing to watch: how does splitting the document by heading change which answer the agent retrieves when two sections both use similar terms?

Build E — Stress and Ship

Run all three probes. Log the result as **HELD** (correct answer), **REFUSED** (clean "I don't have that"), or **MISSED** (wrong or hallucinated answer).

PROBE	QUERY	RESULT
Synonym	"What's your no-egg option for celebration cakes?"	
Out-of-scope	"Can you tell me if the birthday boy will like this flavour?"	
Exact-code	"What does Custom Order Policy clause 3 say about same-day orders?"	

Persona 2 — Allergy Parent

You're buying pastries for your child who has a severe nut allergy. You need to know exactly what's safe before ordering. You're also curious whether the eggless option affects the allergen risk in any way.

Your starter document: `Lumiere_KB.md`

Build A — Feed the Brain

Upload `Lumiere_KB.md` to your Claude Project (or attach to a fresh chat). Then ask:

"Does the almond croissant contain nuts?"

Log the answer here:

Build B — Make It Cite

Open Reference Sheet 1. Copy the **Grounding Prompt** and paste it into your Project's custom instructions (or as the first message of your chat). Then re-ask your Build A question.

The answer should now end with a real **(Source: ...)** line naming a specific heading.

Paste the source line the agent returned:

(Source: _____)

Build C — Make It Refuse

Ask your agent:

"Is your kitchen completely nut-free and certified allergen-free?"

Note: the document states the kitchen is NOT nut-free — so the agent should answer that part from the document. "Certified allergen-free" is the out-of-scope part. Observe whether your agent handles both in the same response, or conflates them.

Before adding the refusal line — what did the agent say?

Open Reference Sheet 1. Copy the **Refusal Line** and add it to your grounding prompt. Re-ask the same question.

After — what changed?

Build D — Observe and Design

Facilitator demo. Watch the live screen and fill in the KB Design Sheet chunk/tag fields at the end of this workbook based on what you observe.

Key thing to watch: how does tagging the Allergens section separately change the precision of the retrieval?

Build E — Stress and Ship

Run all three probes. Log the result as **HELD**, **REFUSED**, or **MISSED**.

PROBE	QUERY	RESULT
Synonym	"Are there any items safe for someone with a tree-nut sensitivity?"	
Out-of-scope	"Can Lumière guarantee my child won't have a reaction?"	
Exact-code	"What does the Allergens & Ingredients section say about cross-contamination?"	

Persona 3 — Corporate Bulk

You're organising a company event at your Andheri office — 80 people, mixed dietary needs including a group who need eggless options, Friday delivery.

Your starter document: `Lumiere_KB.md`

Build A — Feed the Brain

Upload `Lumiere_KB.md` to your Claude Project (or attach to a fresh chat). Then ask:

"What's the minimum order for corporate catering, and do you deliver to Andheri?"

Log the answer here:

Build B — Make It Cite

Open Reference Sheet 1. Copy the **Grounding Prompt** and paste it into your Project's custom instructions (or as the first message of your chat). Then re-ask your Build A question.

The answer should now end with a real **(Source: ...)** line naming a specific heading.

Paste the source line the agent returned:

(Source: _____)

Build C — Make It Refuse

Ask your agent a question that is outside the document:

"Can you process a company GST invoice directly through this chat?"

Before adding the refusal line — what did the agent say?

Open Reference Sheet 1. Copy the **Refusal Line** and add it to your grounding prompt. Re-ask the same question.

After — what did the agent say?

Build D — Observe and Design

Facilitator demo. Watch the live screen and fill in the KB Design Sheet chunk/tag fields at the end of this workbook based on what you observe.

Key thing to watch: how does the Delivery section's structure affect what the agent returns for a branch-specific query?

Build E — Stress and Ship

Run all three probes. Log the result as **HELD**, **REFUSED**, or **MISSED**.

PROBE	QUERY	RESULT
Synonym	"Do you handle large office orders?"	
Out-of-scope	"What's the catering market rate in Mumbai this year?"	
Exact-code	"What is the delivery pincode range for the Andheri West branch?"	

Persona 4 — Delivery Edge-case

You live in Sector 90, Gurgaon and you've just heard Lumière is expanding. You want to know if you can get delivery — and if a Gurgaon branch opens, whether their eggless cakes will be available there too.

Your starter document: `Lumiere_KB.md`

Build A — Feed the Brain

Upload `Lumiere_KB.md` to your Claude Project (or attach to a fresh chat). Then ask:

"Do you deliver to Sector 90, Gurgaon?"

The Version 1 document covers Mumbai only — there is no Gurgaon entry yet. The agent should say so cleanly rather than guessing.

Log the answer here:

Build B — Make It Cite

Open Reference Sheet 1. Copy the **Grounding Prompt** and paste it into your Project's custom instructions (or as the first message of your chat). Then re-ask your Build A question.

The answer should now end with a real (**Source: ...**) line naming a specific heading.

Paste the source line the agent returned:

(Source: _____)

Build C — Make It Refuse

Ask your agent a question that is outside the document:

"Can you tell me when you'll open a branch in Delhi?"

Before adding the refusal line — what did the agent say?

Open Reference Sheet 1. Copy the **Refusal Line** and add it to your grounding prompt. Re-ask the same question.

After — what did the agent say?

Build D — Observe and Design

Facilitator demo. Watch the live screen and fill in the KB Design Sheet chunk/tag fields at the end of this workbook based on what you observe.

During the demo the facilitator will add a Gurgaon branch entry to the live document and re-upload it. Note how the agent's answer to the Gurgaon delivery question changes after the document update.

Build E — Stress and Ship

Run all three probes. Log the result as **HELD**, **REFUSED**, or **MISSED**.

For the synonym probe: re-ask after the facilitator has added the Gurgaon branch entry to the live document — the answer should change.

PROBE	QUERY	RESULT
Synonym	"Which areas near DLF Garden City can you deliver to?" (re-ask after doc update)	
Out-of-scope	"Can you GPS-track my delivery in real time?"	
Exact-code	"What are the pincodes covered by the Powai branch delivery zone?"	

Reference Sheet 1 — Grounding Prompt Library

The grounding prompt (Build B)

Paste this into your Project's custom instructions, or as the first message of your chat.

```
You are a knowledge agent for Lumière Bakery. Answer questions using ONLY the document(s) in your knowledge. After every answer, name the section, heading, or page you pulled it from, like this: (Source: <section/heading>). Do not add outside facts.
```

The refusal line (Build C)

Add this after the grounding prompt once you've confirmed citation is working.

```
If the answer is not in the document, say: 'I don't have that in my knowledge base' – and stop. Never guess, never fill in from outside knowledge.
```

Combined prompt (copy-paste ready for Steps B and C together)

Paste the grounding prompt above, then immediately add the refusal line below it. Together they form the complete custom instructions for your Lumière Bakery agent.

Sharpening tips

- If the source line comes back vague ("Source: the document"), add: "Name the exact heading or page number, never just 'the document'."
- If the agent guesses even after the refusal line, tighten the test: ask something more cleanly outside the doc (e.g. future expansion plans, live competitor pricing).
- If you adapt this for a different document, swap "Lumière Bakery" for the name of your organisation or system.

Reference Sheet 2 — The RAG Dial

RAG is a dial, not a switch. Find where your use-case sits today; write the level on the headline row of your Knowledge Base Design Sheet.

Level 0 — No retrieval. The model answers from its training data alone. It guesses Lumière's lead time rather than reading the document. The baseline you move off of.

Level 1 — CAG (Cache-Augmented Generation). Load the whole document into context — no vector database, no chunk-and-search. The model has the entire `Lumiere_KB.md` in front of it. This is Build A. Right for small, stable docs.

Level 2 — Vanilla RAG. Lumière grows to 500 locations and a 10,000-page operations manual. You can't load it all, so you retrieve only the sections that match the question and answer from those. The default — roughly eight in ten business cases land here. This is Builds B through D.

Level 3 — Hybrid RAG. Keyword and semantic search run together. "Custom Order Clause 3" (exact keyword) and "eggless celebration cake for a birthday" (semantic meaning) both fire in the same query. Reach for it when your documents contain clause numbers, SKUs, or policy codes alongside plain prose. This is Build E.

Level 4 — Agentic RAG. The agent decides for itself when and how often to retrieve, across more than one source — for example, comparing this season's offer codes to last season's and flagging what changed. Concept today; Session 5 picks it up.

Name-drops (one line each): - **GraphRAG** — builds a graph of relationships across documents; useful for cross-document synthesis. Complementary to plain RAG, not a replacement. - **Re-ranking RAG** — retrieve 20 candidate chunks, re-sort to the best 3 before answering; useful for precision-critical queries where the first-retrieved chunk is rarely the best one.

Reference Sheet 3 — Knowledge Base Design Sheet

Fillable sheet

KNOWLEDGE BASE DESIGN SHEET

Name: _____ Persona: _____

Document grounded: Lumiere_KB.md Branch: _____

★ HEADLINE ROW — Where on the RAG dial?

Level: _____ Why: _____

1. CHUNK STRATEGY — how I'd split this document (by section / by topic / by branch):

2. METADATA TAGS

a. Structural tags — section labels retrieval needs
(e.g. [CustomOrders], [Allergens], [Delivery]):

b. Semantic tags — questions each section answers
(e.g. "Can I get a same-day cake?"):

3. QUERY RANGE

This agent SHOULD answer: _____

This agent must REFUSE: _____

4. GROUNDING + REFUSAL PROMPT — my final custom-instructions text:

Worked example — Last-minute Birthday persona

```
Name: [Name]                Persona: Last-minute Birthday
Document grounded: Lumiere_KB.md

★ HEADLINE ROW: Level-1 CAG — one doc, stable content, fits whole in Project.

1. CHUNK STRATEGY: by section heading (each of the 8 sections is one chunk).

2. METADATA TAGS:
  a. Structural: [CustomOrders] [Allergens] [Delivery] [Offers] [Hours]
  b. Semantic: "Can I get same-day?" "Is the eggless option available?"
              "Do you deliver to X?" "Does LUMIERE15 apply to eggless cakes?"

3. QUERY RANGE:
  SHOULD: cake availability, eggless surcharge, lead times, delivery zones,
          allergen info, current offers including eggless eligibility.
  REFUSE: competitor pricing, revenue/financial data, GPS-live tracking.

4. GROUNDING + REFUSAL PROMPT:
  [Grounding Prompt from Reference Sheet 1]
  [Refusal Line from Reference Sheet 1]
```

Troubleshooting

1. **Vague source** — agent returns "(Source: the document)" rather than a real heading. Fix: add "Name the exact heading or page number, never just 'the document'."
2. **Still guesses after the refusal guardrail** — ask a question that is more cleanly outside the doc (expansion timelines, competitor data, anything not written in `Lumiere_KB.md`).
3. **Retrieval miss — fact you know is in the doc but the agent can't find it** — the section probably lacks a clear heading or tag. Add a heading, add a tag, re-upload, re-ask.
4. **Good retrieval but vague response** — the agent found the right section but paraphrased loosely. This is an integration failure, not a retrieval miss. Sharpen the instruction: "Quote the exact clause or sentence; do not paraphrase."

Build Lumière yourself — two prompts, two stages

You watched the Lumière demo end-to-end. Now you can rebuild it. Two prompts. Each one is a single message you paste into a fresh Claude chat. The first prompt produces a grounded bot. The second turns it into an agent. Everything you've learned in Sessions 1–4 is in here: how a chatbot fails (S1), why you'd want an agent (S2), prompt engineering (S3), grounded retrieval and refusal (S4).

Before you start

Install Ollama from ollama.com/download. In a terminal, run `ollama pull qwen2.5:3b-instruct` once and leave Ollama running in the background. That's the model the bot will use — small, fast, runs entirely on your laptop, no API keys, nothing leaves your machine.

Create a folder. Put `Lumiere_KB.md` in it. Save the HTML each prompt produces into that same folder as `index.html`. To run it: open a terminal in the folder and run `python3 -m http.server 8000`. Then visit `http://localhost:8000` in your browser. (The local server is needed so the page can read `Lumiere_KB.md`. Double-clicking the HTML won't work — browsers block file reads from `file://` for security.)

If a prompt's output doesn't work, paste the error back into the same chat and ask Claude to fix it. Don't start over.

Stage 1 — A grounded bakery bot

This prompt produces a small Lumière website with a chatbot in the corner. The bot reads the bakery's knowledge base from a file in the same folder, answers customer questions strictly from it, cites the section it pulled from, and refuses anything not in the document.

Paste this into a fresh Claude chat. (No attachment needed — the bot fetches the file at runtime.) Send.

Build me one single index.html file: a small website for "Lumière", a Mumbai artisan bakery (three branches: Bandra West, Andheri West, Powai), plus a floating chatbot in the corner that answers customer questions about it.

The chatbot must talk to my local Ollama:

```
POST http://localhost:11434/api/chat
body: { "model": "qwen2.5:3b-instruct", "messages": .[.], "stream": false }
reply is at data.message.content
```

The bot's brain is a file called Lumiere_KB.md in the SAME FOLDER as index.html. When the page loads, fetch('Lumiere_KB.md') and store the text in a variable called KB_TEXT. If the fetch fails, disable the chatbot and show "Couldn't load Lumiere_KB.md – make sure it's next to this file and that you opened the page via a local server (not by double-clicking)."

Build the system prompt every turn like this:

You are Lumière bakery's customer-service bot.

Rules:

1. Answer ONLY from the KNOWLEDGE below. Never use outside facts.
2. End every answer with (Source: <section name>).
3. If the answer is not in KNOWLEDGE, reply exactly:
"I don't have that in my knowledge base." – and stop. Never guess.

KNOWLEDGE:

<KB_TEXT>

Look: warm cream background, terracotta accent, serif headings. Floating chat button bottom-right; clicking it opens a chat panel with a message log, an input, a send button. Show "thinking..." while waiting. If the Ollama fetch fails, say "Ollama isn't running – start it and try again."

Use textContent (not innerHTML) for all user/bot text – no script injection.

One file. No frameworks. No build step. Output the complete index.html only, no commentary.

Save the output as `index.html`, run the local server, open it, click the chat button, and ask "what are your opening hours?" The reply should end with `(Source: Locations & Hours)`. Ask "what's your CEO's salary?" and it should refuse with the exact canonical sentence. If both work, Stage 1 is done.

Stage 2 — Make it agentic

The Stage 1 bot is a *retriever* — it reads the document and answers. Stage 2 makes it an *agent* for one specific question: "do you deliver to me?"

For that question, the bot must not guess from the document. It must:

1. Ask the browser for the user's location.

2. Read the branch list from the document (so newly opened branches are picked up automatically).
3. Calculate the distance from the user to the nearest branch.
4. Apply the bakery's delivery rule and answer with a real distance and ETA.

This is the move from S2 — *a chatbot that can use tools, then decide what to say from the results.*

Paste this into the SAME Claude chat where Stage 1 was built. Send.

```
Upgrade the bot: when the user asks about delivery / "near me" / "my area",
skip Ollama and run this tool chain instead.

Tools:
1. getMyLocation() → browser geolocation → {lat, lng}.
   If denied: "I need your location – please allow it and ask again."
2. listBranches() → parse KB_TEXT for branches under "Locations & Hours".
   For each, use the coordinates if present, else geocode the address once via
   https://nominatim.openstreetmap.org/search?q=<address>&format=json&limit=1
   and cache the result.
3. nearest(lat, lng) → haversine distance from user to each branch; return
   the closest { branch, km }.

Rule – Lumière delivers within 7 km. ETA: ≤2 km → 20 min · 2-5 km → 30 min ·
5-7 km → 45 min · >7 km → refuse + suggest self-collect.

Reply in plain English naming the branch + distance + ETA, end with
(Source: live location + Lumière KB).

Show a collapsible "Agent steps" panel under each delivery reply listing
each tool call and its result.

Do NOT call Ollama for delivery answers. All other questions still use the
Stage 1 grounded flow, unchanged. Use textContent for dynamic text. One file.
Output the complete updated index.html only.
```

Save, refresh, ask "do you deliver to my place?" Allow the location permission. The bot should reply with the nearest branch + distance + ETA. Click open the "Agent steps" panel below the reply to see every tool that ran. That panel is the lesson made visible.

What the tools actually do — read this before you wonder

The Stage 2 prompt names three tools. Each is a small plain-JavaScript function that does one thing. They are called in a fixed order; nothing magical happens; the model is not what calls them. Your code's `if` statement decides "is this a delivery question?" — and if yes, the tools run.

#	TOOL	WHAT IT DOES	WHERE THE DATA COMES FROM	WHAT CAN GO WRONG
1	<code>getMyLocation()</code>	Returns the user's latitude and longitude	The browser's built-in geolocation API. The first time it runs, the browser asks the user for permission.	User declines permission → the tool returns an error → the agent replies asking them to enable it.
2	<code>listBranches()</code>	Returns every Lumière branch the document knows about, with coordinates	Reads <code>KB_TEXT</code> (the bakery's knowledge file) and pulls out anything under the "Locations & Hours" section. If a branch only has an address (no coordinates), it geocodes the address once via OpenStreetMap and caches the result.	OpenStreetMap rate-limits if asked too often → cache means each branch is geocoded only once per page load.
3	<code>nearest(lat, lng)</code>	Picks the closest branch to the user and returns its name and distance in kilometres	Calls <code>listBranches()</code> for the branch list, then runs the haversine formula (the standard great-circle-distance equation) for each.	Nothing — pure maths.

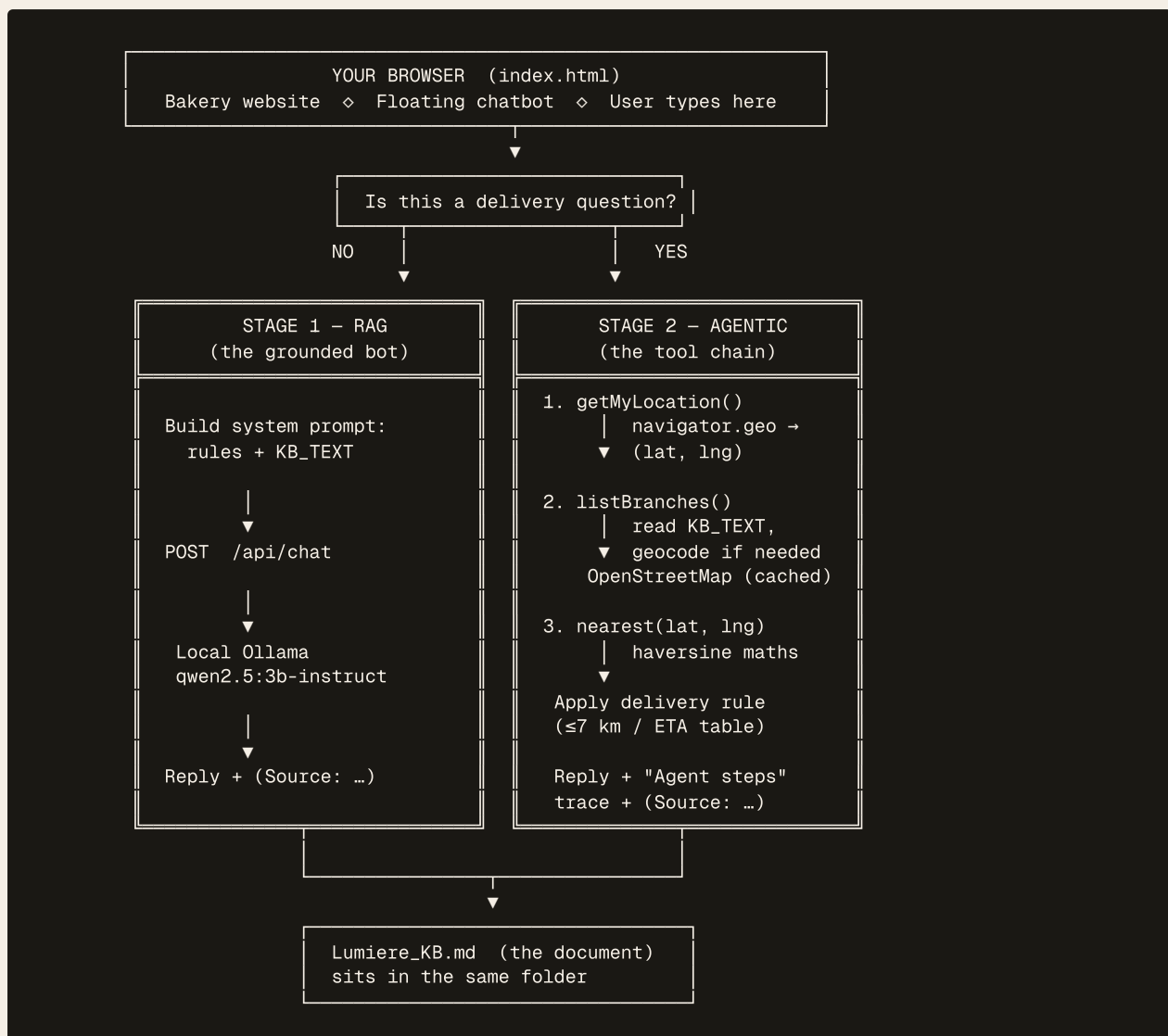
After the three tools run, the agent applies the **delivery rule** in JavaScript (not in the model):

- ≤ 2 km → ~20 min delivery
- 2–5 km → ~30 min
- 5–7 km → ~45 min
- More than 7 km → refuse, suggest self-collect

It then writes the reply in plain English naming the branch, the distance, and the ETA — and ends with `(Source: live location + Lumière KB)` so the user can see *exactly* where that answer came from.

Why isn't Ollama answering this question? Because the model doesn't know your location and can't do geolocation. The model is great at reading documents and writing in natural language. It is the wrong tool for "where am I and how far is the bakery?" That's a factual lookup the browser and basic geometry can do directly. **The agent uses the model where the model is good, and uses tools where tools are good.** That's the whole idea.

Architecture — both stages, side by side



Read the diagram top-down for any question, and you can answer "what just happened?" yourself:

- A normal question (hours, menu, allergens) → left branch → Ollama with the whole document loaded in the prompt → grounded reply with a (Source:...) line.
- A delivery question → right branch → three tools run in order → JavaScript applies the bakery's delivery rule → reply with branch + distance + ETA, plus a visible trace of each tool call.

Same chat window. Same user experience. Two completely different paths under the hood, picked by one `if` statement.

What you have, end to end

A single HTML file. A real working customer agent. Open the website, ask about hours, menu, allergens, custom orders — the bot answers from the document and cites it. Ask something not in the document — it refuses honestly. Ask about delivery — the agent uses your live location, reads

the branch list from the document (so adding a new branch to the doc is the whole "deploy" step), computes the distance, and tells you the ETA.

Two prompts. One model. Three tools. The S4 lesson made into something you can show your team on Monday.

What's Next — Session 5

Session 5 covers Agent Architecture — how agents think, remember, and act. Today's agent retrieves when you ask it to; you drove every query. Session 5 gives it the will to decide for itself when to retrieve, which source to consult, and what to do with the result. The brain you built today stays; next session we add the will to use it.